Computational Statistics and Simulation: Section 4. Markov Chain Monte Carlo methods

Master in Mathematics and Applications

Isabel Pereira,
isabel.pereira@ua.pt

Universidade de Aveiro,
2025/26

Section 4: Markov Chain Monte Carlo (MCMC) methods

1. Markov Chain concepts
2. Metropolis-Hastings algorithm
3. Gibbs sampling
4. Monitoring convergence

1. Markov Chain concepts

We saw earlier in Section 2 that simulating samples of a given univariate distribution can present some complexity, if the function distribution does not have a known explicit form.

In the case where the objective is to simulate observations from a multivariate model

$$\mathbf{X} = (X_1, \ldots, X_k)$$

the problem becomes even more complicated and the computational process can be quite time consuming.

At the outset, a possible approach consists of phasing the generation process through a marginal and successive conditionals considering the factorization

$$F_{\mathbf{X}}(x_1, \ldots, x_k) = F_{x_1}(x_1) F_{X_2|X_1}(x_2) \ldots F_{X_k|x_1,\ldots,x_{k-1}}(x_k)$$

which in practice can turn out to be an inefficient process.

1. Markov Chain concepts

Monte Carlo simulation methods via Markov Chains (MCMC) constitute a class of algorithms to simulate complex multivariate distributions. They are based on the construction of a homogeneous Markov chain whose distribution is the limit of the distribution to be simulated.

The Metropolis-Hastings algorithm and, in particular, the Gibbs are some of these methods.

1. Markov Chain concepts

Let us now make a fundamental change in the choice of our simulation strategy.

- So far, we have typically generated *iid* variables
- The Metropolis-Hastings algorithm generates *correlated* variables, from a Markov Chain

The use of Markov chains expands our range of applications

- The requirements about the target function f are minimal
- Decomposition and aware of high-dimensional problems
  in a succession of more basic problems

Definition 1.1: Markov chain
A Markov chain $\{X_t, t = 0, 1, 2, 3 \ldots\}$ is a sequence of dependent r.v.'s, in which the probability function of X_t only depends on X_{t-1}, i.e.

$$P(X_t = x_t | X_{t-1}, \ldots, X_2, X_1) = P(X_t = x_t | X_{t-1})$$

Definition 1.2: Transition Kernel/transition probab.
The conditional distribution of $X_t | X_{t-1}$ is given by the transition kernel K; we have

$$X_{t+1} | X_t, \ldots, X_2, X_1 \sim K(X_{t+1} | X_t)$$

Example 1.1: random walk

$$X_{t+1} = X_t + \epsilon_t, \epsilon_t \sim N(0,1)$$

independent of X_t; the Markov transition probabilities corresponds to the pdf of $N(X_t, 1)$

MCMC Markov Chains typically have a property of very strong stability - stationary probability distribution:

- Probability distribution f such as , if $X_t \sim f$ then $X_{t+1} \sim f$, taking up

$$\int_{\chi} K(y|x) f(x) dx = f(y)$$

1. Markov Chain concepts

- MCMC Markov chain are irreducible
  - The K Kernel allows movements over all the state space
  - For any X_0, the sequence $\{X_t\}$ has a positive probability of visiting any state space region.
- MCMC Markov chain are recorrent
  - Returns infinite times to any state
- In recorrent chains, the stationary distribution is the limit distribution, as well.
- If f is the limit probability distribution:

$$X_t \to X \sim f, for \quad any \quad initial \quad value \quad X_0 \quad (ergodicity)$$

- For integrable functions h, if exists $E[h(X)]$

$$\frac{1}{T}\sum_{t=1}^{T} h(X_t) \xrightarrow{P} E[h(X)], T \to \infty$$

(Large Numbers Law, or Ergodic Theorem)

2. Metropolis-Hastings algorithm

The basis of MCMC methods is simple

- Given a target density f
  - build a Markov kernel with stationary distribution f
  - generate a Markov chain $\{X_t\} \to X \sim f$
  - the integrals are approximated by the Ergodic theorem
- Metropolis-Hastings algorithm is an example of this procedure
  - give the target density f, simulate from the candidate $q(y|x)$
  - generate a Markov chain $\{X_t\} \to X \sim f$
  - it is only necessary to know the ratio $\frac{f(y)}{q(y|x)}$, less than one constant.

Metropolis-Hastings algorithm

Given x_t

1. Generate $Y \sim q(y|x_t)$
2. Calculate

$$X_{t+1} = \begin{cases} Y & with \quad probability \quad \alpha(x_t, Y) \\ x_t & with \quad probability \quad 1 - \alpha(x_t, Y) \end{cases}$$

with $\alpha(x, y) = min\{\frac{f(y)}{f(x)} \frac{q(x|y)}{q(y|x)}, 1\}$

- $q \hookrightarrow$ candidate/proponent distribution
- $\alpha(x, y) \hookrightarrow$ Metropolis-Hastings acceptance probability

Example 2.1: Generate values from a Beta (2.7,6.3) using M-H

- target density f: beta(2.7,6.3)
- proponent q: Uniforme(0,1)

At the end, compare with the direct generation of RPN's with the R function rbeta - using histograms and overlaying the pdf.

Metropolis-Hastings algorithm

1. Consider x_0 such as: $f(x_0) > 0$, keep it in $x[1] = x_0$
2. Repeat for $t = 1, 2, \ldots, T-1$ (i=1,2, ..., T):
   (a) Generate y from $q(y|x_t) \sim Unif(0,1)$, with $x_t = x[i-1]$
   (b) Generate u from $Unif(0,1)$
   (c) Calculate $\alpha(x_t, y)$, using $f \hookrightarrow$ beta(2.7, 6.3) pdf and $q \hookrightarrow$ Unif(0,1) pdf
      - If $u \leqslant \alpha(x_t, y)$ accept y and take $x_{t+1} = y$;
      - Otherwise, $x_{t+1} = x_t$;
      - Keep x_{t+1} in $x[i] = x_{t+1}$
   (d) Increment t and return to 2.

Remarks

1. Step (c) can be done using the instruction

$$x_{t+1} = x_t + (y - x_t)\mathbf{1}_{\{u \leqslant \alpha(x_t, y)\}}$$

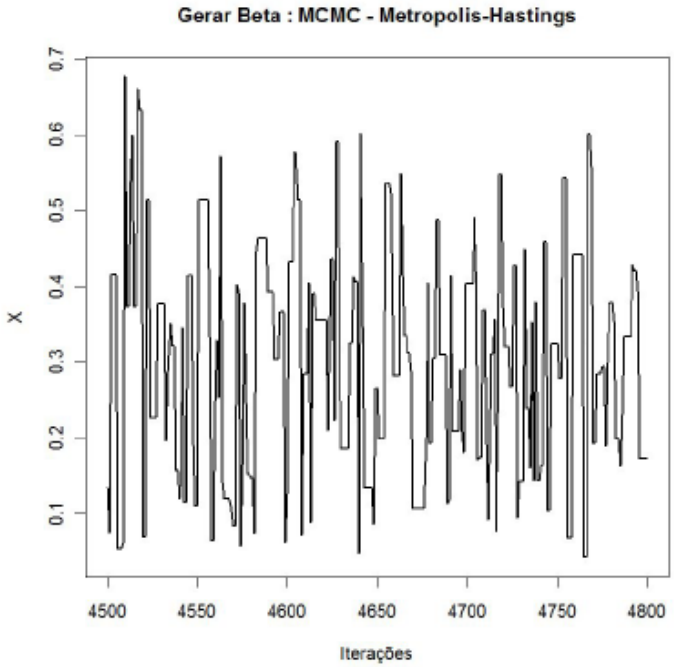
2. In practice, it is necessary to wait for the chain to reach its equilibrium state: must "throw away"a certain number of initial realizations of the chain (called the heating or burn-in period)

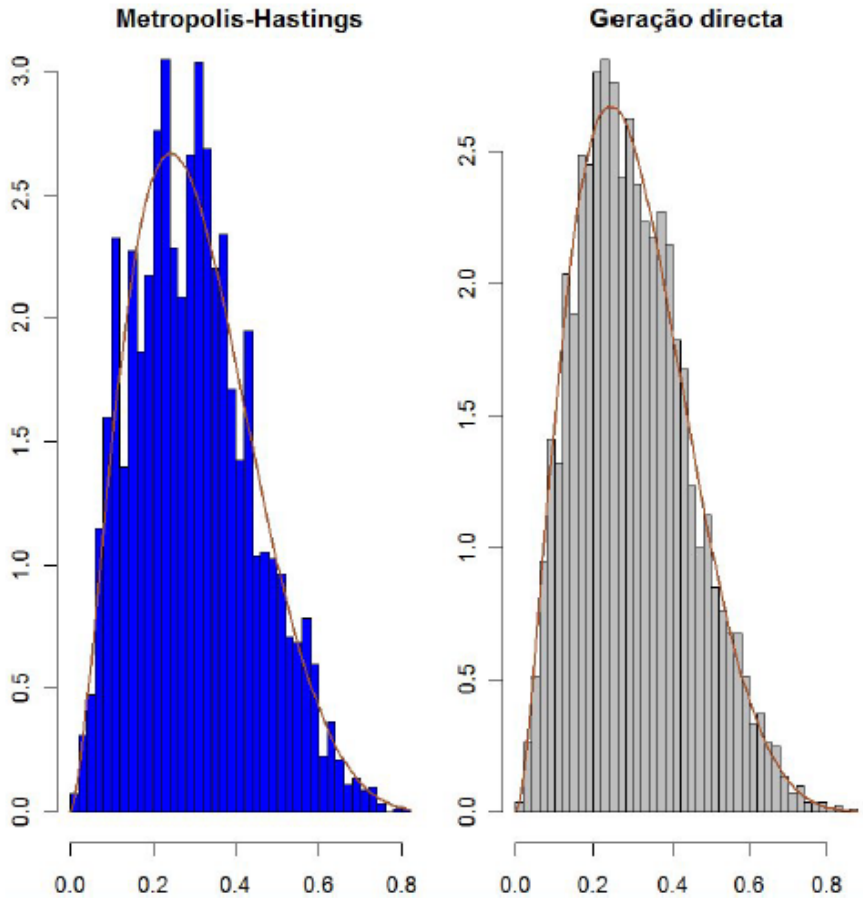
3. The value x_0 is any as long as it is in the support of the target function. Since the candidate must have the same support as target, so one way to initialize the chain is to generate a value of the candidate.

Generating values of Beta distribution, using M-H algorithm

```
a=2.7; b=6.3;# valores iniciais
nsim=5000
X=rep(runif(1),nsim)  # inicializar a cadeia
for (i in 2:nsim){
   Y=runif(1)
   alpha=min(dbeta(Y,a,b)/dbeta(X[i-1],a,b),1)
   X[i]=X[i-1] + (Y-X[i-1])*(runif(1)<alpha)
   }
plot(4500:4800,X[4500:4800],ty="l",lwd=2,xlab="Iterações",ylab="X")
title(main="Gerar Beta : MCMC - Metropolis-Hastings")
#-----------------Histograms----------------------
par(mfrow=c(1,2),mar=c(2,2,1,1))
hist(X[1000:nsim],breaks=50,col="blue",main="Metropolis-Hastings",fre=FALSE)
curve(dbeta(x,a,b),col="sienna",lwd=2,add=TRUE)
hist(rbeta(4000,a,b),breaks=50,col="grey",main="Geração directa",fre=FALSE)
curve(dbeta(x,a,b),col="sienna",lwd=2,add=TRUE)
```

Generating values of Beta distribution, using M-H algorithm (cont)





Remarks

In the last example

1. on the one hand, the pdf of the Uniform(0,1) candidate model, $q(.|x_t)$, is constant (=1) and therefore does not depend on x_t ;
2. on the other hand, in the implementation of the algorithm through the instruction

$$x_{t+1} = x_t + (y - x_t)\mathbf{1}_{\{u \leqslant \alpha(x_t, y)\}}$$

we do not have access to the number of rejections of y as observations to be attributed to x_{t+1} and, therefore, there is no idea of the method efficiency.

Let us now exemplify the MH Method in order to illustrate exactly these topics:

Example 2.2: Generate values from a Rayleigh

$$f(x) = \frac{x}{\sigma^2} e^{-x^2/(2\sigma^2)}, x \leqslant 0, (\sigma > 0)$$

(a) Use for candidate $q(.|x_t)$ the chi-square with X_t df, $\chi^2_{X_t}$. In the implementation of the algorithm, output the information about the percentage of rejections of y candidates, for a run of $T = 10000$.

(b) Overlay the Rayleigh pdf on the histogram of the generated observations, discarding the first 2000 observations of the Markov chain (burn in sample).Compare the empirical with the theoretical quantiles using a QQplot.
$x_q := F^{-1}(q) = \sigma - 2log(1-q)^{1/2}, 0 < q < 1$

Generating values of Raileigh distribution

exercise (a)

```
## fdp da Rayleigh
f <- function(x, sigma) {
    if (any(x < 0)) return (0)
    stopifnot(sigma > 0)
    return((x / sigma^2) * exp(-x^2 / (2*sigma^2)))
}

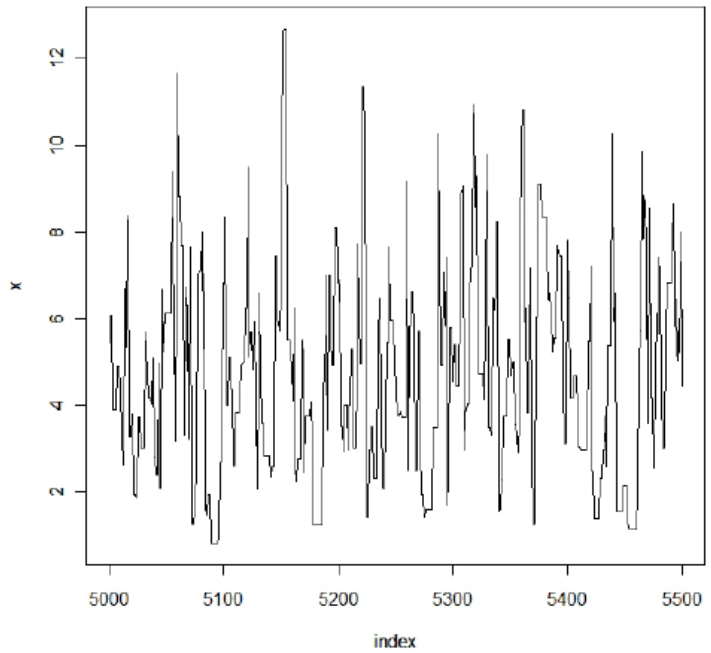
T <- 10000      # número de corridas
sigma <- 4      # escolha de um valor para sigma
x <- numeric(T)
x[1] <- rchisq(1, df=1)  # valor inicial X_0
k <- 0          # regista o número de rejeições
u <- runif(T)   # Gera os T valores NPA's U(0,1)

for (i in 2:T) {
    xt <- x[i-1]
    y <- rchisq(1, df = xt)    # valor candidato
    num <- f(y, sigma) * dchisq(xt, df = y)
    den <- f(xt, sigma) * dchisq(y, df = xt)
    if (u[i] <= min(num/den,1)) x[i] <- y else {
         x[i] <- xt
         k <- k+1      # incrementa k porque y foi rejeitado
         }
    }

>   print(k)
[1] 3996
```

Representation of a portion of the Markov chain generated:

```
index <- 5000:5500
y1 <- x[index]
plot(index, y1, type="l", main="", ylab="x")
```



exercise (b): Comparison of theoretical and empirical distributions

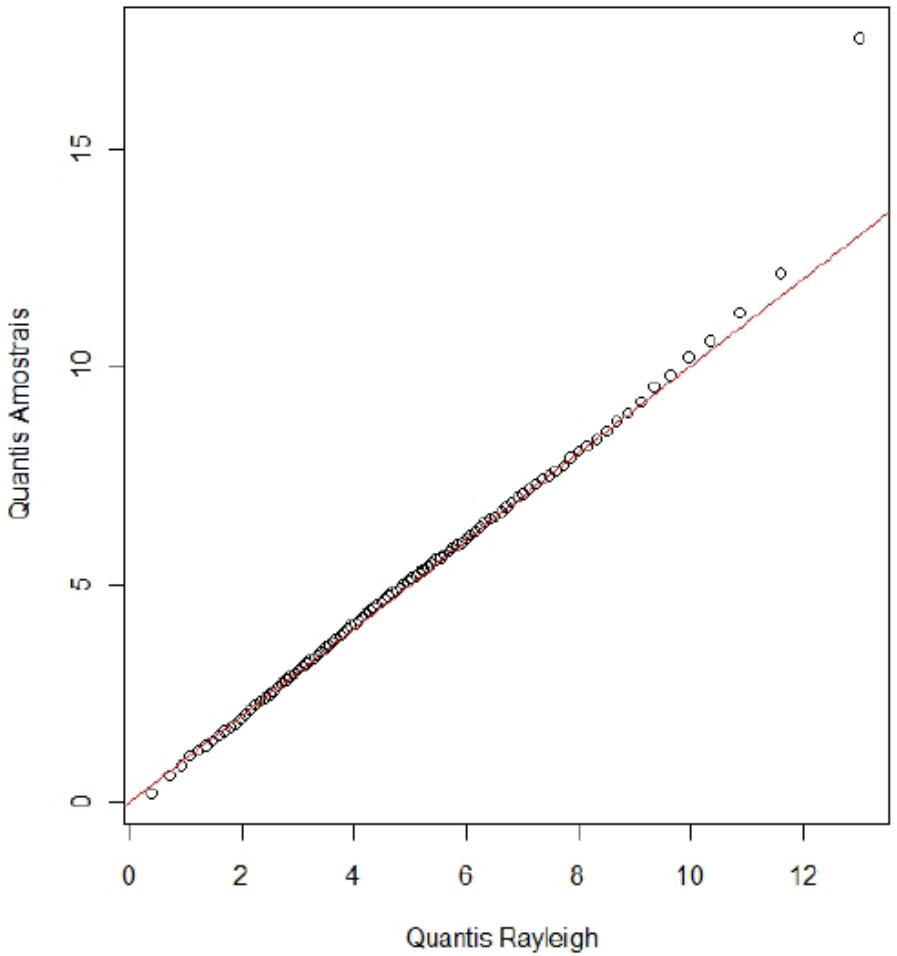
```
b <- 2001       #  burn in sample fora
y <- x[b:T]
a <- ppoints(100)
QR <- sigma * sqrt(-2 * log(1 - a))  # quantis da Rayleigh

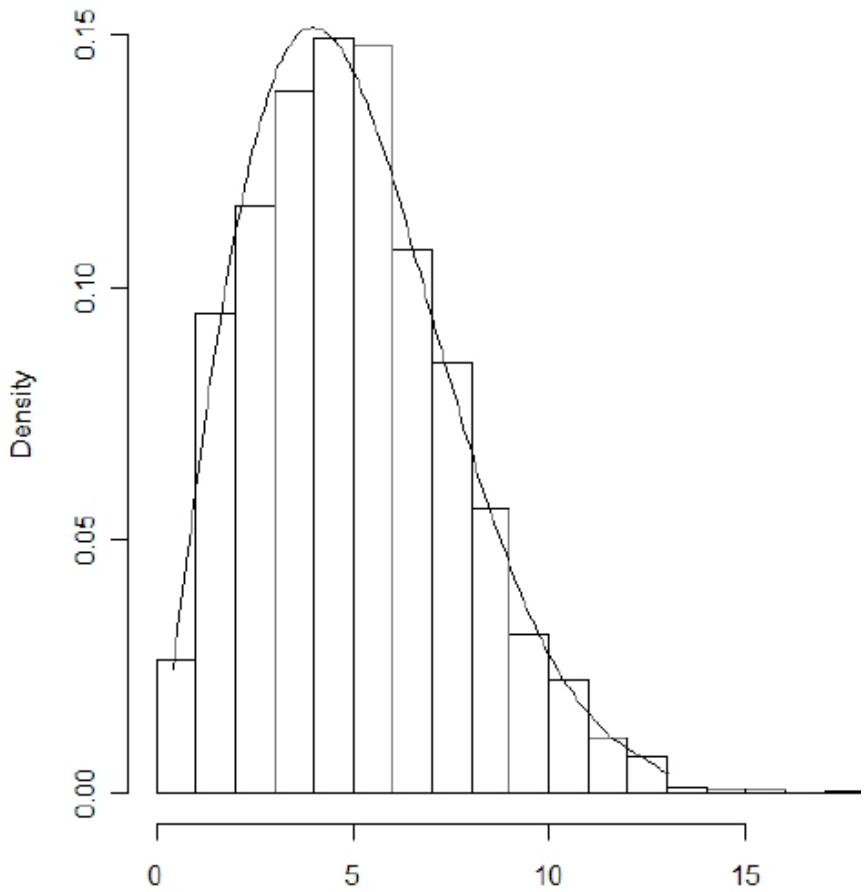
par(mfrow=c(1,2))
qqplot(QR, y, main="",
    xlab="Quantis Rayleigh ", ylab="Quantis Amostrais")
abline(0, 1,col="red")

hist(y, main="", xlab="", freq=FALSE)
lines(QR, f(QR, 4))
```

2. Metropolis-Hastings algorithm

exercise (b): Comparison of theoretical and empirical distributions





It seems to be confirmation of the sample values with the theoretical ones

Important Remarks

1. the M-H method only depends on the ratios

$$\frac{f(y)}{f(x_t)} \quad and \quad \frac{q(x_t|y)}{q(y|x_t)}$$

(which are independent of normalizing constants)

2. You must "run"the algorithm on dispersed x_0 starting points to verify the convergence in the produced results

1. **Independent Metropolis-Hastings algorithm**

   A special case is the one obtained by choosing y candidates that are independent of x, i.e. using the transition probability of the form $q(y|x) = q(y)$, then

$$\alpha(x_t, y) = min\{\frac{f(y)q(x_t)}{f(x_t)q(y)}, 1\}$$

2. **Metropolis-Hastings algorithm when proponent is symmetric**

   In the case of candidate's symmetry, ie when $q(x|y) = q(y|x)$

$$\alpha(x_t, y) = min\{\frac{f(y)}{f(x_t}, 1\}$$

   so the probability of acceptance is independent of q

Metropolis-Hastings Algorithm with Random Walk

Another variant is based on the idea of simulating Y_t according to a random walk, for each t:

$$Y_t = X_t + \epsilon_t$$

- $\epsilon_t \hookrightarrow$ random perturbance with pdf g
- $g \hookrightarrow$ independent of X_t (Uniform, normal,t-Student, etc ...)
- $g \hookrightarrow$ symmetric about zero, ie, $g(-z) = g(z)$
- the candidate density has the form $q(y|x) = g(y - x)$
- The Markov chain associated with q is a random walk

Metropolis-Hastings Algorithm with Random Walk

Given x_t

1. Generate $y \sim g(y - x_t)$
2. Calculate

$$X_{t+1} = \begin{cases} Y & with \quad probability \quad \alpha(x_t, Y) = min\{\frac{f(Y)}{f(x_t)}, 1\} \\ x_t & with \quad otherwise \end{cases}$$

Remark:

$$\alpha(x_t, y) = min\{\frac{f(y)q(x_t|y)}{f(x_t)q(y|x_t)}, 1\} = min\{\frac{f(y)g(x_t - y)}{f(x_t)g(y - x_t)}, 1\} = min\{\frac{f(y)}{f(x_t)}, 1\},$$

since

$$g(x_t - y) = g(y - x_t)$$

Metropolis-Hastings Algorithm with Random Walk

Notes:

- Different choices of g lead to different acceptance rates
- On a random walk, the highest acceptance rate is not always the best strategy (Robert & Casella, 2010)
  - Golden rule for the acceptance rate: 50% (one dimensional case)
- The classic example is generating an $N(0,1)$ from:
  - $Y_t = X_{t-1} + \epsilon_t$
  - $\alpha(x_t, y) = min\{exp\{(x_t^2 - y^2)/2\}, 1\}$
- If $\epsilon_t \sim U(-\delta, \delta), \delta$ controls the acceptance rate

3. Gibbs algorithm

- Gibbs sampling is a special case of Metropolis-Hastings, with particular interest in
  - Bayesian Statistic
  - Computational Statistics
- usually exist two Gibbs sampling types
  - two-stage
  - multi-stage

3. Gibbs algorithm: two-stage

Two-stage Gibbs sampling creates a Markov chain from a joint distribution

Consider X and Y random variables with joint probability $f(x, y)$

- $f_{Y|X}(y) \hookrightarrow$ conditional distribution of Y given X
- $f_{X|Y}(x) \hookrightarrow$ conditional distribution of X given Y

In particular, Gibbs sampling generates the Markov chain (X_t, Y_t) according to the following steps

Two-stage Gibbs algorithm

Initialize with $X_0 = x_0$

For $t = 1, 2, ..., N$, generate

- $Y_t \sim f_{Y|X=x_{t-1}}(y_t)$
- $X_t \sim f_{X|Y=y_t}(x_t)$

3. Gibbs algorithm: two-stage

Remarks

- The algorithm is easy to implement as long as you know both conditionals
- The stationary distribution is $f(x, y)$
- The convergence of the Markov chain is assured since the supports of the conditionals are connected

Example 3.1: Gibbs sampling for bivariate normal

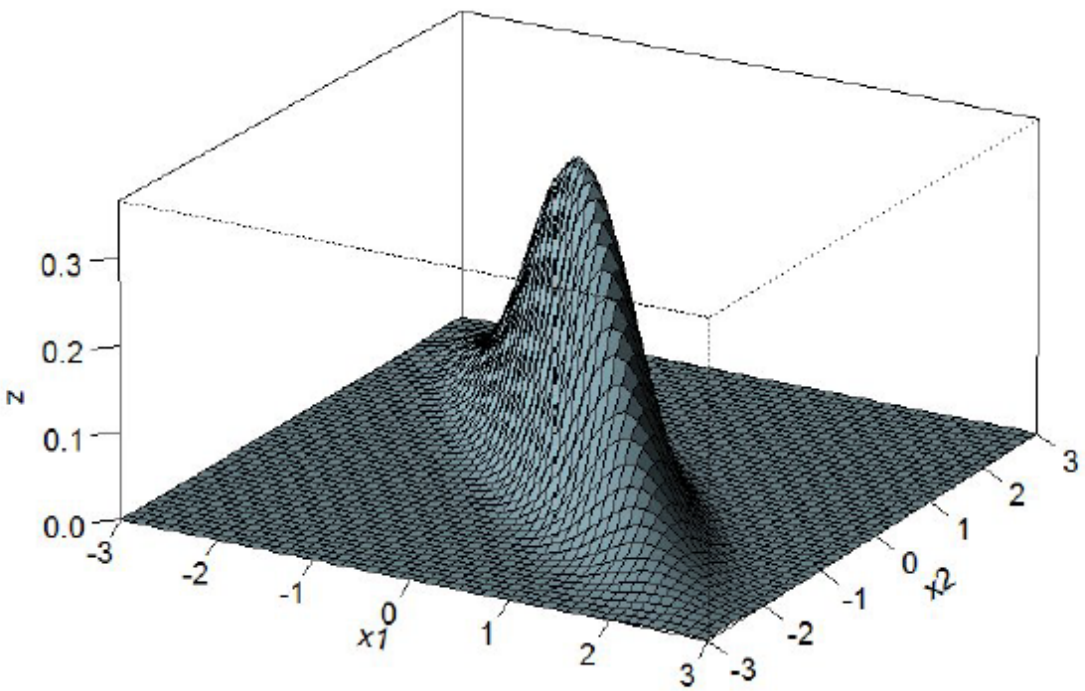
$$(X, Y) \sim N_2(\mathbf{0}, \Sigma), \mathbf{0} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}.$$

Gibbs algorithm:
Given x_t, generate

- y_{t+1} from $Y_{t+1}|x_t \sim N(\rho x_t, 1 - \rho^2)$
- x_{t+1} from $X_{t+1}|y_{t+1} \sim N(\rho y_{t+1}, 1 - \rho^2)$

Bivariate Normal distribution

$\mu_1 = 0, \mu_2 = 0, \sigma_{11} = 1, \sigma_{22} = 1, \sigma_{12} = 0.9, \rho = 0.9$



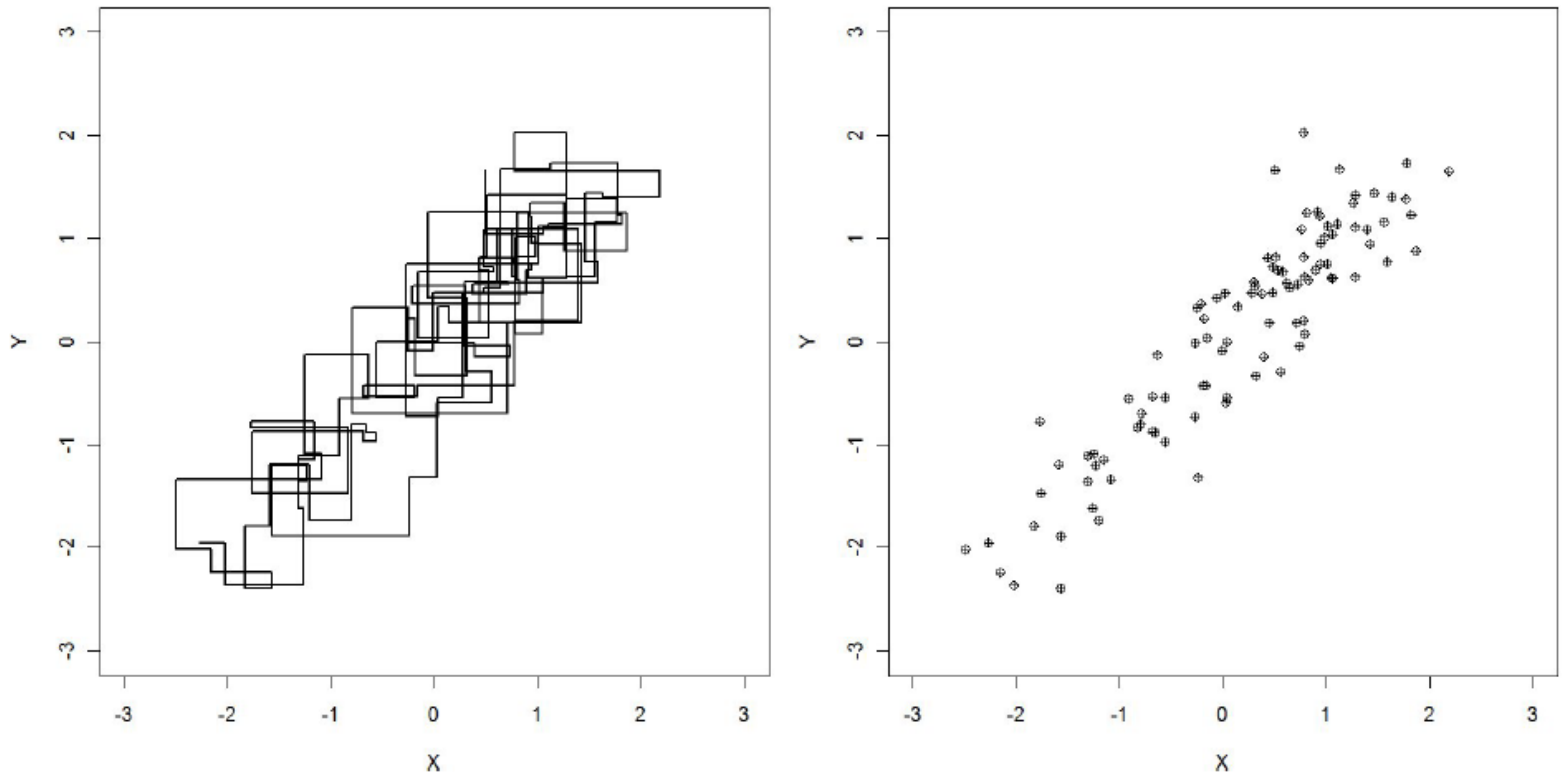
$$f(\mathbf{x}) = \frac{1}{2\pi\sqrt{\sigma_{11}\,\sigma_{22}\,(1-\rho^2)}} \cdot \exp\left\{-\frac{1}{2(1-\rho^2)}\left[\frac{(x_1-\mu_1)^2}{\sigma_{11}} - 2\,\rho\,\frac{x_1-\mu_1}{\sqrt{\sigma_{11}}}\,\frac{x_2-\mu_2}{\sqrt{\sigma_{22}}} + \frac{(x_2-\mu_2)^2}{\sigma_{22}}\right]\right.$$

3. Gibbs algorithm: two-stage

Plot of the Gibbs trajectory and scatter plot of (X_t, Y_t) for the bivariate normal (100 iterations)

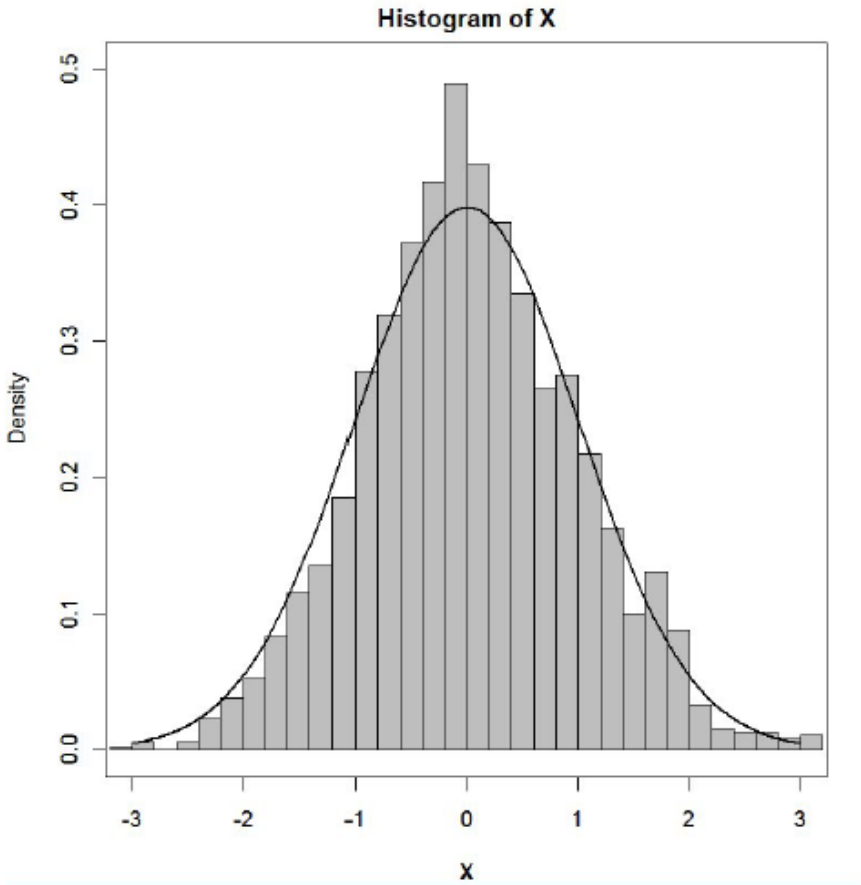
```
## Gibbs bietapa -- Normal bivariada com rho=0.9 ------##
nsim<-100;
X=array(0,dim=c(nsim,1))
Y=array(0,dim=c(nsim,1))
rho=.9
Std=sqrt(1-rho^2)
X[1]=rnorm(1) # inicializar a cadeia
Y[1]=rnorm(1) # inicializar a cadeia
for(i in 2:nsim)
{
X[i]=rnorm(1,mean=rho*Y[i-1],sd=Std)
Y[i]=rnorm(1,mean=rho*X[i],sd=Std)
}
#----------Plot da trajectória do Gibbs -----------------#
par(mfrow=c(1,2),mar=c(4,4,2,1))
plot(X,Y,pch="",xlim=c(-3,3),,ylim=c(-3,3))
for(i in 1:nsim)
lines(c(X[i],X[i]),c(Y[i],Y[i+1]),lwd=2)   # movimentos verticais
for(i in 1:nsim)
lines(c(X[i],X[i+1]),c(Y[i+1],Y[i+1]),lwd=2) # movimentos horizontais
plot(X,Y,pch=10,xlim=c(-3,3),,ylim=c(-3,3)) # núvem de pontos (x_t,y_t)
```

3. Gibbs algorithm: two-stage



3. Gibbs algorithm: two-stage

```
#--------histograma  com 2000 iterações ------------------------
hist(X,freq=F,xlim=c(-3,3),ylim=c(0,.5),col="gray",breaks=25)
par(new=T)
plot(function(x)dnorm(x),xlim=c(-3,3),ylim=c(0,.5),lwd=2,ylab="")
```



- A subcadeia $\{X_t\}$ satistaz $X_{t+1}|X_t = x_t \sim \mathcal{N}(\rho^2 x_t, 1-\rho^4)$
- Recursivamente

$$X_{t+1}|X_0 = x_0 \sim \mathcal{N}(\rho^{2t} x_0, 1-\rho^{4t}) \to N(0,1)$$

- Com número de iterações = 10000

```
> var(X)
          [,1]
[1,] 1.009051
> var(Y)
          [,1]
[1,] 1.025963
> cor(X,Y)
           [,1]
[1,] 0.9005668
```

- obteve-se convergência para a *distribuição conjunta* e para ambas *distribuições marginais*

Hierarchical Models in Bayesian Analysis- Beta-Binomial model

A hierarchical model specifies the joint distribution through successive strata of conditional distributions

Example 3.2: Beta-Binomial distribution

Consider the hierarquical model

$$X|\theta \sim Bin(n, \theta)$$

$$\theta \sim Beta(a, b)$$

Marginal distribution X is the Beta-Binomial

3. Gibbs algorithm: two-stage, example

- Joint distribution

$$f(x,\theta) = C_x^n \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \theta^{x+a-1}(1-\theta)^{n-x+b-1}$$

- Full conditionals
  - $X|\theta \sim Bin(n,\theta)$
  - $\theta|X \sim Beta(X+a, n-X+b)$

- the marginal distribution of X is the Beta-Binomial

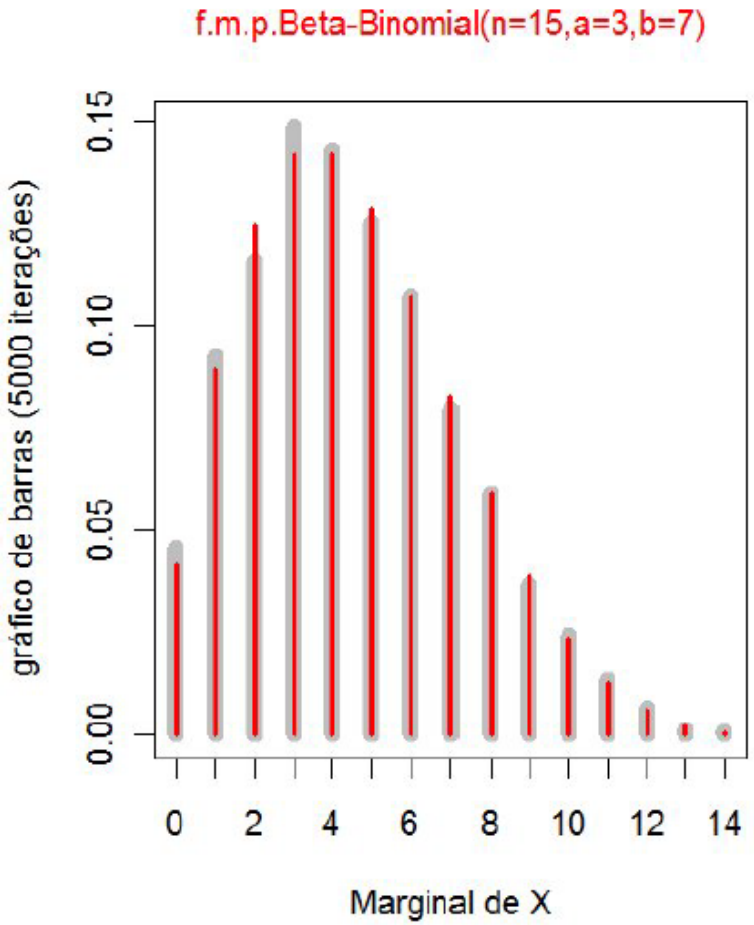
$$m(x) = \int_0^1 C_x^n \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \theta^{x+a-1}(1-\theta)^{n-x+b-1} d\theta$$

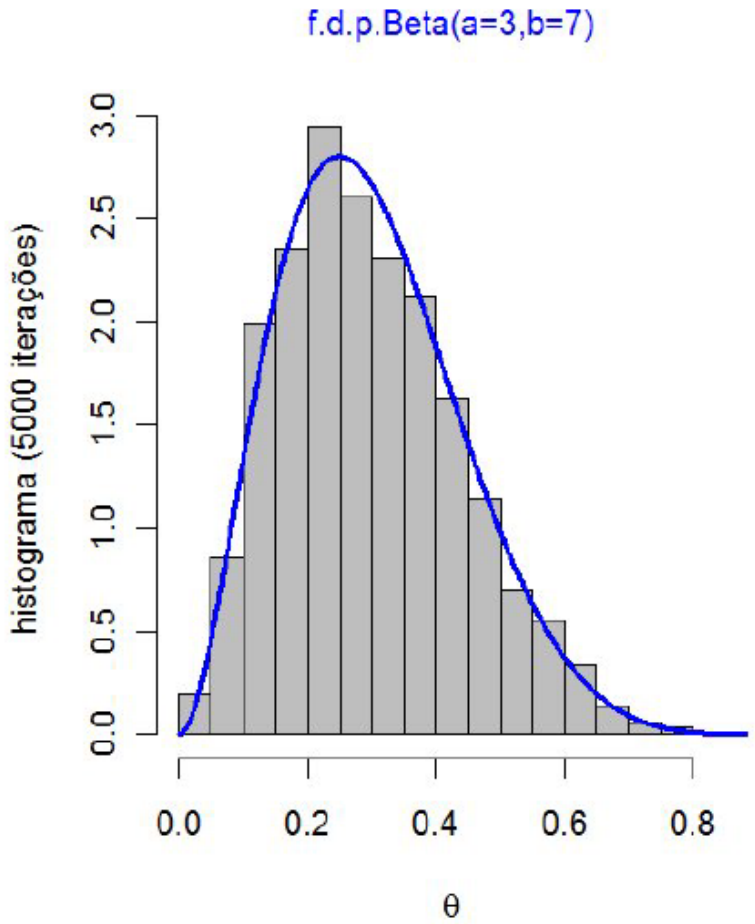
$$m(x) = C_x^n \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \frac{\Gamma(x+a)\Gamma(n-x+b)}{\Gamma(n+a+b)}$$

3. Gibbs algorithm: two-stage

```
betabi<- function(x,a,b,n) {
px<-choose(n, x)*gamma(a+b)/(gamma(a)*gamma(b))*gamma(x+a)*gamma(n-x+b)/gamma(n+a+b)
return(px)
}
##  Beta-Binomial  ########################################
 Nsim=5000 # iterações
 n=15
 a=3
 b=7
 X=T=array(0,dim=c(Nsim,1)) # valores iniciais para os arrays
 T[1]=rbeta(1,a,b) #valores iniciais para as cadeias
 X[1]=rbinom(1,n,T[1])
 for (i in 2:Nsim){
 X[i]=rbinom(1,n,T[i-1]) #----------------- Amostrador X[i] | teta[i-1]
 T[i]=rbeta(1,a+X[i],n-X[i]+b)  # ----------Amostrador  teta[i] | X[i]
 }
 ## ---------------------GRáficos   ---------------------------------
par(mfrow=c(1,2))
#------------------------ marginal de X -----------------------------
plot(table(X)/Nsim,xlab="Marginal de X",
ylab="gráfico de barras (5000 iterações)",cex=1.5, lwd=8,col="grey")
x<-seq(0,n)
px<-betabi(x,a,b,n)
lines(x,px,type="h", lwd=2.5,col="red",main="")
title(main = list("f.m.p.Beta-Binomial(n=15,a=3,b=7)",
 cex=1, col="red", font=1))
#--------------------- marginal de  teta ----------------------------
hist(T,xlab=bquote(theta),ylab="histograma (5000 iterações)",
col="grey",breaks=25,freq=F,main="")
teta<-seq(0,1,by=0.001)
lines(teta,dbeta(teta,a,b),col="blue", lwd=2.5,type="l")
title(main = list("f.d.p.Beta(a=3,b=7)", cex=1, col="blue", font=1))
```

3. Gibbs algorithm: two-stage





Example 3.3: Normal hierarquical model

A study of metabolism in 15-year-old girls led to the following results about energy consumption, measured over a 24-hour period.

$y : 91, 504, 557, 609, 693, 727, 764, 803, 857, 929, 970, 1043, 1089, 1195, 1384, 1713$
Model:

$$X := log(Y_i) \sim N(\theta, \sigma^2), i = 1, \ldots, n$$

with $\theta \sim N(\theta_0, \tau^2)$ and $\sigma^2 \sim IG(a, b)$ where IG(a,b) is the inverted gamma, defined by a rv Z with p.d.f.

$$f(z) = \frac{e^{-b/z} b^a}{z^{a+1}\Gamma(a)}$$

Use Gibbs sampler to study the posterior probabilities for θ and σ^2, making the appropriate plots for each parameters.

Example 3.3: Normal hierarquical model(cont)
The posterior distribution is proportional to the joint dist. of $(\underline{x}, \theta, \sigma^2)$, meaning

$$\pi(\theta, \sigma^2|\underline{x}) \propto f(\theta, \sigma^2, \underline{x}) = f(\underline{x}|\theta, \sigma^2)\pi(\theta)\pi(\sigma^2)$$

ie,

$$\pi(\theta, \sigma^2|\underline{x}) \propto [\frac{1}{(\sigma^2)^{n/2}} e^{-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i-\theta)^2}][\frac{1}{\tau} e^{-\frac{1}{2\tau^2}(\theta-\theta_0)^2}][\frac{1}{(\sigma^2)^{a+1}} e^{-\frac{b}{\sigma^2}}]$$

Full conditionals:

$$\theta|\sigma^2, \underline{x} \sim N(\frac{\sigma^2}{\sigma^2+n\tau^2}\theta_0 + \frac{n\tau^2}{\sigma^2+n\tau^2}\bar{x}, \frac{\tau^2\sigma^2}{\sigma^2+n\tau^2})$$

$$\sigma^2|\theta, \underline{x} \sim IG(\frac{n}{2}+a, \frac{1}{2}\sum_{i=1}^{n}(x_i-\theta)^2 + b)$$

Notice that to generate values from Inverted Gamma dist. we use the property:
If $Z \sim IG(a,b) \Rightarrow Y = 1/Z \sim Gamma(a,b)$

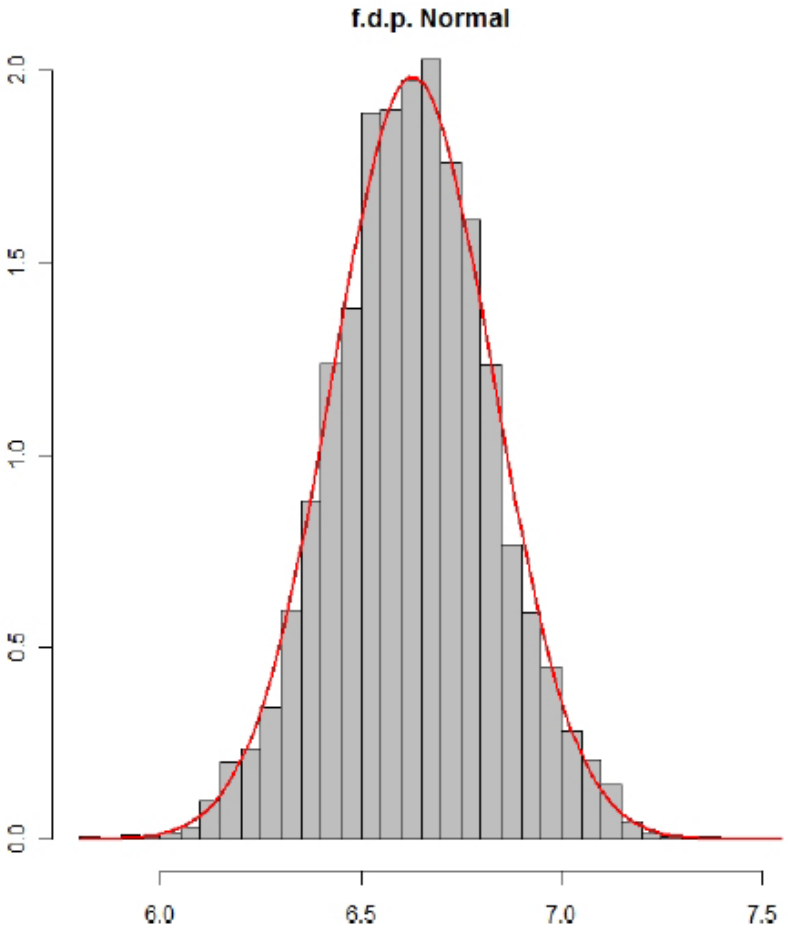
3. Gibbs algorithm: two-stage

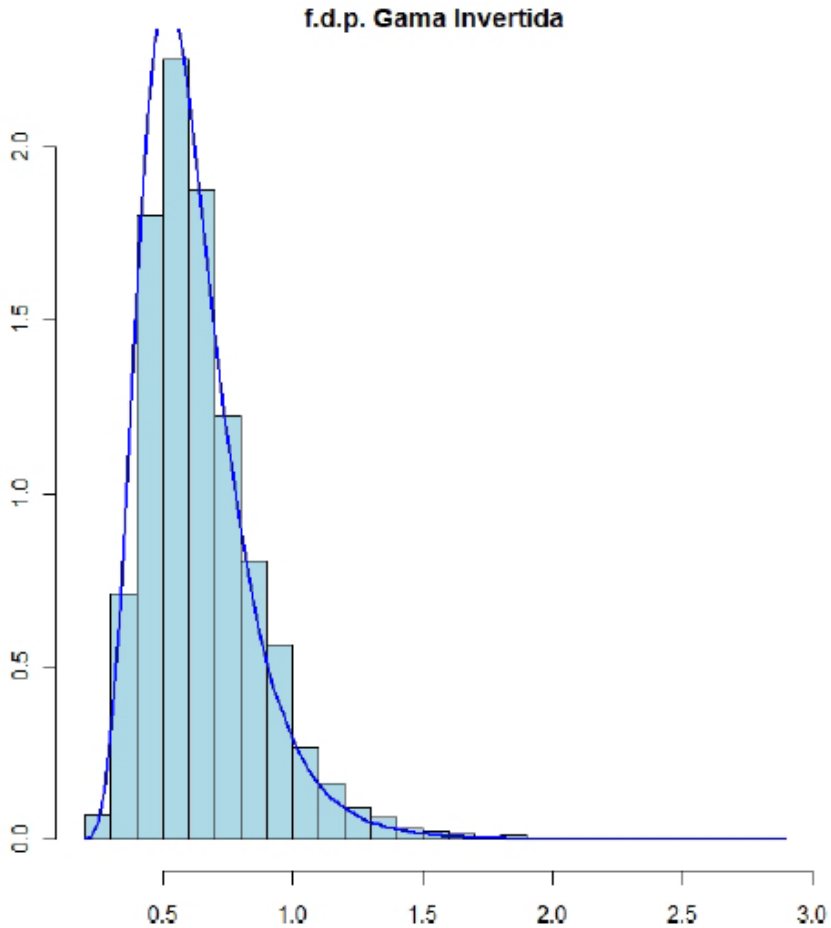
```
#--------modelo Normal Hierárquico---------------
x<-c(91,504,557,609,693,727,764,803,857,929,970,1043,1089,1195,1384,1713)
x<-log(x) # logaritmizar os dados
x
mean(x)
n<-length(x)
#--------------Inicializar theta e sigma2-------------------------
nsim<-5000
theta<-array(mean(x), c(nsim,1))
sigma2<-array(var(x),c(nsim,1))
theta0<-6 ; tau2<-10; a<-3;b<-3
#-----------------------------------------------------------------------
for(i in 2:nsim)
{
## ---------valor médio e variância da teta-> ------Normal(mt,vt)---------
mt<-(theta0*sigma2[i-1]/(sigma2[i-1]+n*tau2))+(n*tau2*mean(x)/(sigma2[i-1]+n*tau2))
vt<-sigma2[i-1]*tau2/(sigma2[i-1]+n*tau2)
theta[i]<-rnorm(1, mean=mt,sd=sqrt(vt))
## --------- shape e rate da sigma2-> -------------- IG(sh,ra) ------------
sh<-n/2+a
ra<-(sum((x-theta[i])^2)/2+b)
sigma2[i]<-1/rgamma(1,shape=sh,rate=ra)
}
# ------------------------------------------------------
```

3. Gibbs algorithm: two-stage

```
#-------------ESTIMAR teta e sigma2 ---------------------------------
est_theta<- mean(theta);est_sigma2<- mean(sigma2);
est_sigma<- sqrt(est_sigma2);
est_theta;
est_sigma2;
est_sigma
## ------------------------------------------------------------------
print(c(sort(theta)[.05*nsim],sort(theta)[.95*nsim])) # quantis empíricos (5%,95%)
## ----------------  IC 90% para a média ---------------------
print(c(mean(theta)- qnorm(0.95)*sqrt(var(theta)/nsim),
mean(theta)+ qnorm(0.95)*sqrt(var(theta)/nsim)))
#---------------Plot-----------------------------------------
par(mfrow=c(1,2))
hist(theta, xlab=bquote(theta), ylab="5000 iterações",col="grey",breaks=25,freq=F,main="")
title(main=c("f.d.p. Normal"))
curve(dnorm(x,mean=mean(theta)   , sd=sqrt(var(theta))),add=TRUE,col="red",lwd=2)
hist(sigma2, xlab=bquote(sigma^2), ylab="5000 iterações",col="lightblue",breaks=25,freq=F,main="")
title(main=c("f.d.p. Gama Invertida"))
InvGamma <- function (x,a,b) exp(-b/x)*b^a/(x^(a+1)*gamma(a))
sh<-n/2+a
ra<-(sum((x-mean(theta))^2)/2+b)
curve(InvGamma(x,a=sh   , b=ra),add=TRUE,col="blue",lwd=2)
#-------------------------------------------------
```

3. Gibbs algorithm: two-stage





3. Gibbs algorithm: multi-stage

Multi-stage Gibbs sampler creates a Markov chain from a p-dim.joint dist.

Consider a multivariate variable $\underline{X} = (X_1, \ldots, X_p), p > 1$;

Full conditional distributional for $X_i, i = 1, \ldots, p$ is

$$X_i | x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_p \sim f_{X_i | x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_p}(x_i),$$

or $$X_i | x_{(-i)} \sim f_{X_i | x_{(-i)}}(x_i)$$

Multi-stage Gibbs algorithm

Initialize with $\underline{X}^{(0)} = \underline{x}^{(0)}$

For $t = 1, 2, ..., N$, generate

- $X_1^{(t+1)} \sim f_{X_1 | x_2^{(t)}, x_3^{(t)}, \ldots, x_p^{(t)}}(x_1)$
- $X_2^{(t+1)} \sim f_{X_2 | x_1^{(t+1)}, x_3^{(t)}, \ldots, x_p^{(t)}}(x_2)$

$\vdots$

- $X_p^{(t+1)} \sim f_{X_p | x_1^{(t+1)}, x_2^{(t+1)}, \ldots, x_{p-1}^{(t+1)}}(x_p)$

Convergence Monitoring

In MCMC methods basically a succession of iterations is generated

$$\underline{x}^{(0)}, \underline{x}^{(1)}, \ldots, \underline{x}^{(t)}, \ldots$$

which is nothing more than a realization of a Markov chain, for which the theoretical results allow us to conclude that, when

$$t \to +\infty, \underline{x}^{(t)} = (x_1^{(t)}, x_2^{(t)}, \ldots, x_p^{(t)})$$

tends in distribution to a random vector with stationary joint probability density function

$$\pi(\underline{x}) = \pi(x_1, \ldots, x_p)$$

A practical question that arises in MCMC methods is:

- When do you stop the MCMC algorithm?
- What is the size of the "burn in"sample?

Convergence Monitoring

Issues to monitor:

1. *Sampling approximately iid*
2. *Convergence towards stationarity*
3. *Ergodic Convergence for Means*

1. Sampling approximately iid

Actually, the vectors that are being generated are correlated. What strategy to adopt to overcome this problem?

- A possible solution involves building a **long chain using spaced vectors** in order to cancel the correlation effect - **subsampling**
- alternatively, you can proceed with the construction of **several parallel chains**, using only the last vector generated of each of them.

4. Gibbs algorithm: Convergence Monitoring

- In the case of subsampling of lag k what is done is to get a subsample of the long initial chain defined

$$\underline{y}^{(t)} := \underline{x}^{(kt)}$$

which reduces the correlation between two successive points of the resulting chain.

- However, the subsampling process *loses efficiency*. For example, consider two estimators
  1. using all the values of the chain generated

$$\delta_1 = \frac{1}{kT} \sum_{t=1}^{kT} h(\underline{x}^{(t)})$$

  2. using a subsample with lag=k between the generated values

$$\delta_k = \frac{1}{T} \sum_{t=1}^{T} h(\underline{x}^{(kt)})$$

It can be shown that $Var(\delta_1) \leqslant Var(\delta_k)$.

4. Gibbs algorithm: Convergence Monitoring

Therefore, combing previous arguments,

- the length of the complete chain is N
- the burn in period is equal to b initial values
- the subsample considered has lag k between the chosen values

Then, the new subchain

$$\underline{y}^{(n)} := \underline{x}^{(b+kn)}$$

has the sample size n.

This procedure pretends to use values in the stationarity region, reducing the correlation between them.

4. Gibbs algorithm: Convergence Monitoring

2. Convergence towards stationarity

- Evaluate that it is already verified $\underline{x}^{(t)} \sim$ stationary f
  - it is difficult considering just one Markov chain trajectory
- It should be checked
  - independence of the starting point $\underline{x}^{(0)}$, producing several chain realizations for different initial values
- On the other hand, MCMC algorithms suffer from the evil:
  - you've only seen where you've been

  meaning that the support of the distribution stationary should be "all"visited as much as possible
- the typical case in which this is sometimes not detected, is when if you start from an initial value close to the mode of f

3. Ergodic Convergence for Means

Having established the convergence to the stationary distribution, we are confronted with the convergence of empirical averages

$$\frac{1}{T}\sum_{t=1}^{T} h(\underline{x}^{(t)}) \xrightarrow{P} E_{\pi}[h(\underline{X})]$$

for an arbitrary h function.

Additionally, Central Limit Theorem (TLC) may not be applicable.

Graphical diagnosis for stationarity monitoring

- Graphical representations of the density estimates; and monitoring the convergence of the ergodic averages of certain scalar quantities (first and second moments, some percentiles, etc.).
- R programme has coda package with some graphical potentialities
- Download and load library coda
  - mcmc(X)$\hookrightarrow$ Turns an MCMC output (X, vector or matrix) into an MCMC object
  - plot(mcmc(X))$\hookrightarrow$ produces:
    - ⋆ the trajectory of the chain between the successive iterations and
    - ⋆ the non-parametric estimate of the density
  - cumuplot(mcmc(X))$\hookrightarrow$ shows the evolution of the median, 1st. and 3rd. quartiles, throughout the iterations.

Example 4.1: monitoring convergence of MC, with Metropolis-Hastings

Target distribution: $N(0,1)$

proposal distribution: $N(X_t, \sigma^2)$

```
normal.chain <- function(sigma, N, X1) {
        # gera uma cadeia MH para a  Normal(0,1)
        # com candidata Normal(X[t], sigma)
        # valor inicial X1
        x <- rep(0, N)
        x[1] <- X1
        u <- runif(N)
        for (i in 2:N) {
            xt <- x[i-1]
            y <- rnorm(1, xt, sigma) # ponto candidato
            r1 <- dnorm(y, 0, 1) * dnorm(xt, y, sigma)
            r2 <- dnorm(xt, 0, 1) * dnorm(y, xt, sigma)
            r <- r1 / r2
            if (u[i] <= r) x[i] <- y else
                 x[i] <- xt
            }
        return(x)
        }
```

4. Gibbs algorithm: Convergence Monitoring

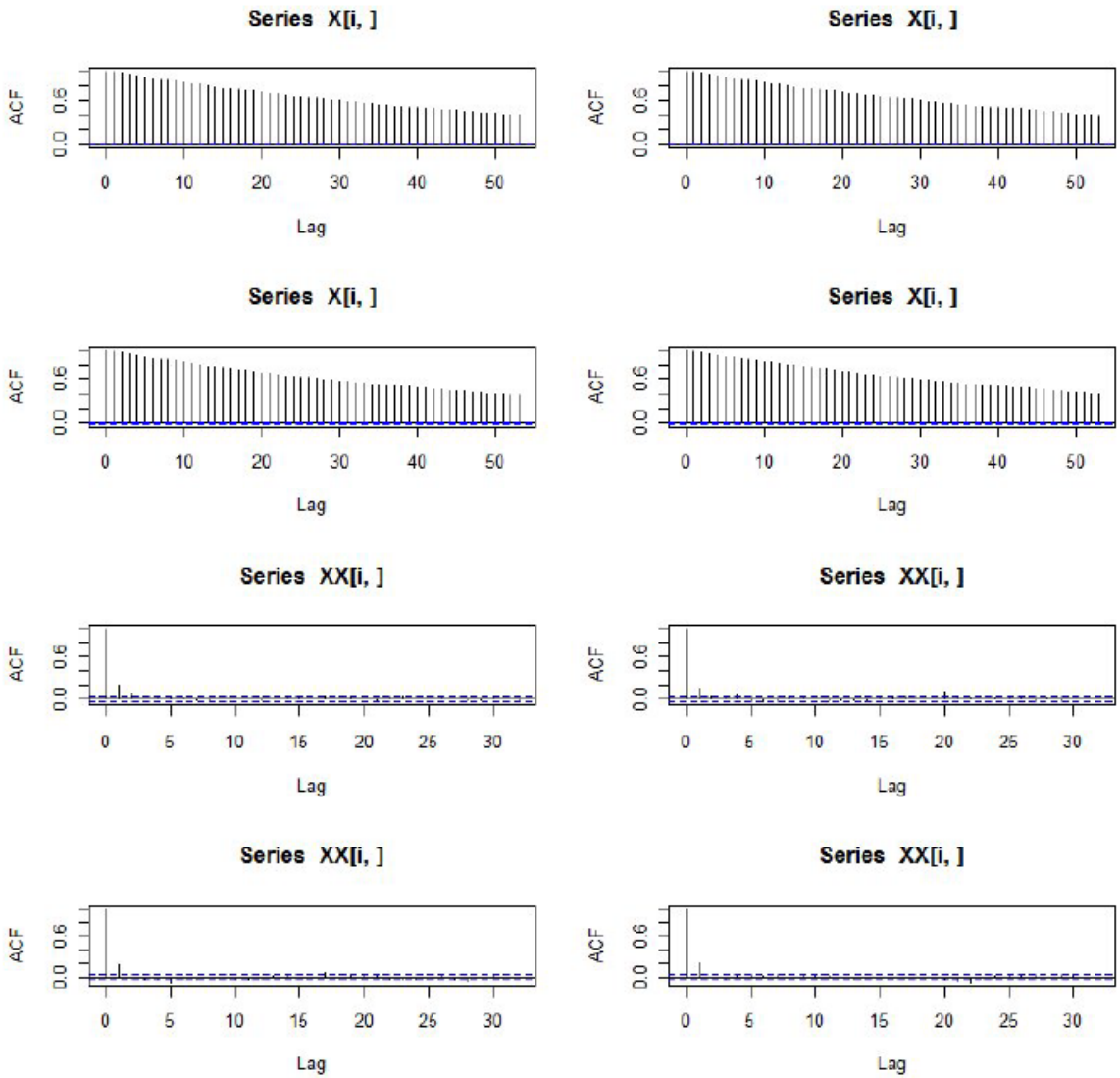
Let's consider

- $k = 4$ chains - 4 dispersed initial values
- $N = 20000$: chain length
- $b = 5000$ burn in period number
- acf: to analyse the correlation between the simulated values
- plot(mcmc(.)): to analyse the trajectories of the chain and the estimated pdf (stationary)
- cumuplot(mcmc(.)): to analyse the evolution of the percentiles

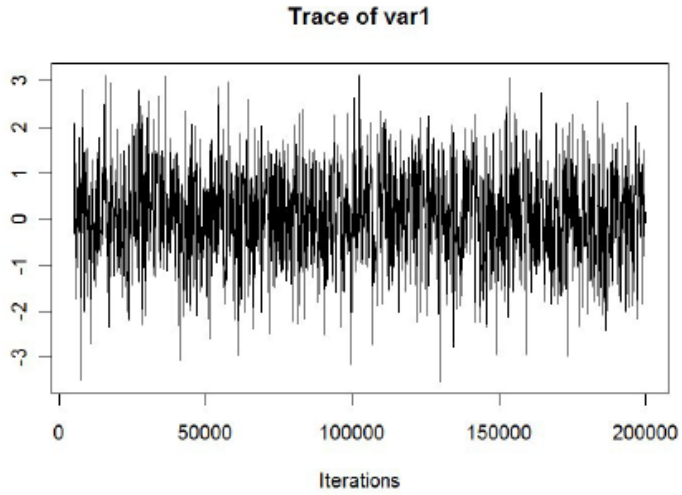
4. Gibbs algorithm: Convergence Monitoring

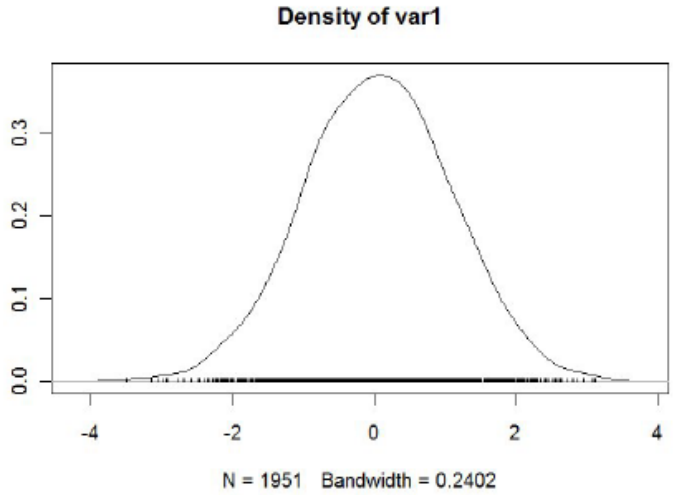
```
   sigma <- .2     # parametro da candidata
    k <- 4          # número de cadeias
    nsim <- 200000    # tamanho da cadeia inicial
    b <- 5000       # burn-in
    x0 <- c(-10, -5, 5, 10) # valores iniciais dispersos
# -------------------X cadeias iniciais
X <- matrix(0, nrow=k, ncol=nsim)
# -------------------XX subcadeias
lag=100
indices=seq(b,nsim,by=lag)
 XX <- matrix(0, nrow=k, ncol=length(indices))
for (i in 1:k){
       X[i, ] <- normal.chain(sigma, nsim, x0[i])
        for(j in 1:length(indices)) {
            XX[i,j ]<-X[i,indices[j]]
        }
}
# ---------------- plot acf das iniciais
par(mfrow=c(4,2))
for (i in 1:k){ acf(X[i, ])
}
# ---------------- plot acf das subcadeias
for (i in 1:k){ acf(XX[i, ])
}
#---------- plot das trajectórias e densidades
plot(mcmc(XX[1, ],start=b,thin=lag))
#---------- plot dos percentis
cumuplot(mcmc(XX[1, ],start=b,thin=lag))
```

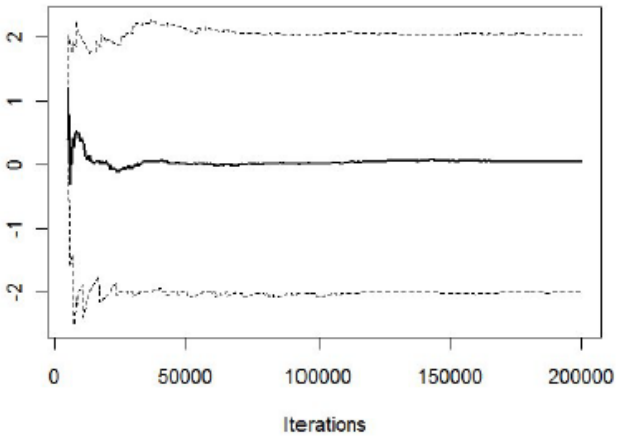
4. Gibbs algorithm: Convergence Monitoring



4. Gibbs algorithm: Convergence Monitoring







Geweke Diagnostic for one chain

Geweke's diagnosis considers the Markov chain of size T,

- two portions, A and B, non-overlapping, (by default, the first $T_A = 10\%T$ and the last $T_B = 50\%T$)
- compares the means of both parts, $\delta_A$and δ_B,
- using a test for the difference of means (null hypothesis H_0)with the objective of investigate whether both parts of the chain have the same distribution
- Test statistic is a normalized Z-score, with standard deviations adjusted for autocorrelation:

$$\frac{\delta_A - \delta_B}{\sqrt{\frac{S_A^2}{T_A}\frac{S_B^2}{T_B}}} \xrightarrow{D} N(0,1)$$

  under the H_0 validity.

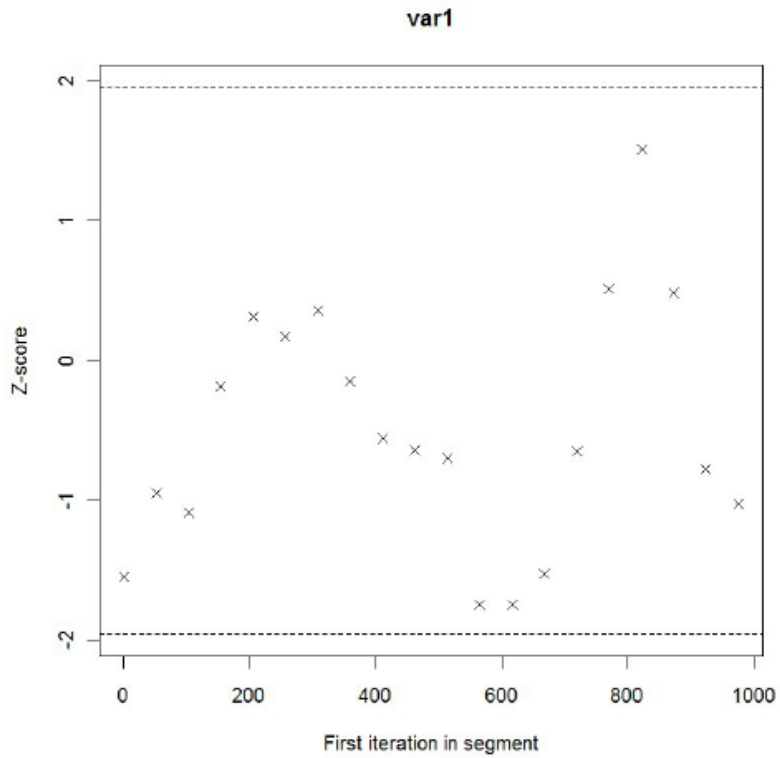
- in R programme, using coda package: geweke.diag and geweke.plot

4. Gibbs algorithm: Convergence Monitoring

```
> geweke.plot(mcmc(XX[1, ]))
> geweke.diag(mcmc(XX[1, ]))

Fraction in 1st window = 0.1
Fraction in 2nd window = 0.5

 var1
-1.55
```



- o plot descarta sucessivos porções iniciais
- o último z-score só usa metade da cadeia
- $z - score = -1.55$ com toda a cadeia inicial (primeiro ponto do plot)

Gelman-Rubin Diagnostic for multiple chains

Gelman-Rubin's diagnosis considers

- M parallel Markov chains of size T, generated from M dispersed initial points

  $$\{\theta_m^{(t)}, 1 \leqslant m \leqslant M, 1 \leqslant t \leqslant T\}$$

  (already discarding other first iterated T)
- compares the variance between the M chains, B_T with the mean of the variances within each of the M chains, W_T (ANOVA type)
- Calculates a reduction scale factor - R_T

  Values of $R_T \approx 1$ suggest that the burn in period can end and the iterative process can continue
- in R programme, using coda package: gelman.diag and gelman.plot

Gelman-Rubin Diagnostic for multiple chains

- Variance between chains

$$B_T = \frac{1}{M-1} \sum_{m=1}^{M} (\bar{\theta}_m - \bar{\theta})^2$$

- mean of the variances inside the chains

$$W_T = \frac{1}{M} \sum_{m=1}^{M} s_m^2, s_m^2 = \frac{1}{T-1} \sum_{t=1}^{T} (\theta_m^{(t)} - \bar{\theta}_m)^2$$

where

  - $\bar{\theta}_m = \frac{1}{T} \sum_{t=1}^{T} \theta_m^{(t)} \hookrightarrow$ mean of the m-th chain
  - $\bar{\theta} = \frac{1}{T} \sum_{t=1}^{T} \bar{\theta}_m \hookrightarrow$ global mean

- in R programme, using coda package: gelman.diag and gelman.plot

Gelman-Rubin Diagnostic for multiple chains

- Variance estimate of the stationary distribution

$$\hat{Var}(\theta) \equiv \hat{\sigma}_T^2 = \frac{T-1}{T} W_T + B_T$$

- Scale reduction factor, R_t

$$R_T^2 = \frac{\hat{\sigma}_T^2 + B_T/M}{W_T} \frac{\upsilon_T + 1}{\upsilon_T + 3}$$

with $\upsilon_T = 2(\hat{\sigma}_T^2 + \frac{B_T}{M})/W_T$

- R_T converges to 1 under stationarity

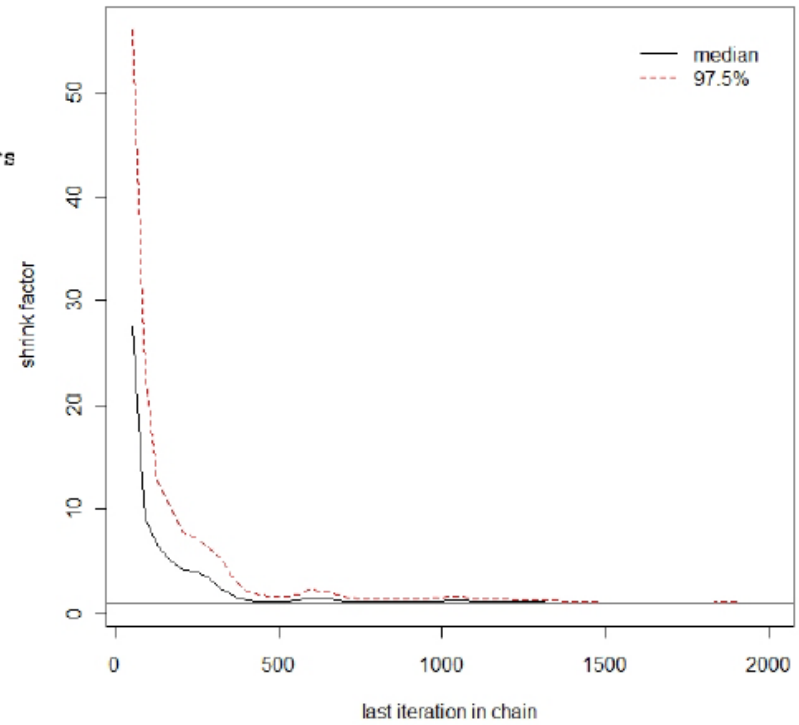
when R_T is large (1.2 or 1.2, suggested by G-R) then we must continue to generate the chains until we improve the convergence to the stationary distribution ($R_T \approx 1$)

Gelman-Rubin Diagnostics

```
library(coda)
sigma <- .2 # parametro da candidata
k <- 4 # número de cadeias
nsim <- 2000 # tamanho da cadeia inicial
b <- 1000 # burn-in inicial
x0 <- c(-10, -5, 5, 10) # valores iniciais dispers
# -------------------X cadeias iniciais
X <- matrix(0, nrow=k, ncol=nsim)
for (i in 1:k){
X[i, ] <- normal.chain(sigma, nsim, x0[i])
}
gelman.plot(mcmc.list(mcmc(X[1, ]),mcmc(X[2, ]),
+ mcmc(X[3, ]),mcmc(X[4, ])))
gelman.diag(mcmc.list(mcmc(X[1, ]),mcmc(X[2, ]),
+ mcmc(X[3, ]),mcmc(X[4, ])))

Potential scale reduction factors:

     Point est. Upper C.I.
[1,]       1.02       1.05
```



sugere convergência da cadeia

References - Bibliography

- Gelman, A. and Rubin, D. (1992) Inference from Iterative Simulation Using Multiple Sequences. Statistical Science 7,457-472.
- Geweke, J. (1992). Evaluation the accuracy of sampling-based approaches to the calculation of posterior moments (with discussion). In Bayesian Statistics 4, J. M Bernardo, M.H. de Groot, D.V. Lindley and A.F.M. Smith, eds, 169-194, University Press, Oxford.
- Robert, C. and Casella, G.(2010). Introducing Monte Carlo methods with R, Springer